

Classroom B - C Lab Worksheet (Single File)

Student Task

Build **one single C file** named `week9_lab_b.c`.

Rules:

1. Use only one `#include <stdio.h>`.
2. Put each question in a separate function.
3. Write only one `main()` and call all question functions inside `main()` in order.
4. Questions 1-5 are normal coding tasks.
5. Questions 6-10 start from intentionally broken code. In each of those questions, fix the broken function before testing it.
6. Everything needed to solve the worksheet is described here. Do not assume any extra starter code.
7. Do not use loops inside the recursive helper functions unless the question explicitly allows it.

Implementation notes:

- You may reuse helper functions across questions.
- Define helper functions before the question functions that call them, or write function prototypes before `main`.
- Do not hard-code the expected output. Your output must come from your functions.
- For functions that return a yes/no result, return `1` for yes/true and `0` for no/false.

Total: 100 points

Each question is worth **10 points**.

Suggested `main()`:

```
int main(void) {
    example1_recursive_sequence_values();
    example2_recursive_number_conversion();
    example3_recursive_array_search();
    example4_recursive_pointer_range();
    example5_recursive_string_printer();
    example6_broken_multiply_r();
    example7_broken_int_division_r();
    example8_broken_array_max_r();
    example9_broken_count_in_array_r();
    example10_broken_print_string_reverse_r();
    return 0;
}
```

Question 1 - Recursive Sequence Values [10 points]

Write recursive functions for Fibonacci numbers, powers, and repeated addition.

Requirements:

1. Write `fibonacci_basic_r(int n)`.
 - Use the course convention: `fibonacci_basic_r(0) = 1, fibonacci_basic_r(1) = 1`.
 - For `n >= 2`, return `fibonacci_basic_r(n - 2) + fibonacci_basic_r(n - 1)`.
2. Write `power_basic_r(int base, int exp)`.
 - Assume `exp >= 0`.
 - Base case: any number to power 0 is 1.
3. Write `multiply_basic_r(int a, int b)` without using the `*` operator.
 - Assume `a >= 0`.
4. Inside `example1_recursive_sequence_values`, test:
 - `fibonacci_basic_r(7)`
 - `power_basic_r(2, 5)`
 - `multiply_basic_r(7, 3)`

Expected output:

```
fibonacci(7): 21
2^5: 32
7 * 3: 21
```

Question 2 - Recursive Number Conversion [10 points]

Use recursion to process and rebuild integer values.

Requirements:

1. Write `sum_digits_basic_r(int n)`.
 - Base case: if `n < 10`, return `n`.
2. Write `count_digits_basic_r(int n)`.
 - Base case: if `n < 10`, return 1.
3. Write `reverse_number_basic_r(int n, int result)`.
 - Each call should move the last digit of `n` to the end of `result`.
 - Base case: if `n == 0`, return `result`.
4. Inside `example2_recursive_number_conversion`, use:

```
int n = 3057;
```

Expected output:

```
number: 3057
digit sum: 15
digit count: 4
reversed: 7503
```

Hint:

- For `reverse_number_basic_r`, the next result can be `result * 10 + n % 10`.

Question 3 - Recursive Array Search [10 points]

Search and count values in an array recursively.

Requirements:

1. Write `search_basic_r(int A[], int size, int n)`.
 - Base case: if `size == 0`, return `0`.
 - If `A[0] == n`, return `1`.
2. Write `index_of_first_basic_r(int A[], int size, int n)`.
 - Return the first index of `n`, or `-1` if it is not found.
 - If the recursive call finds the value in the rest of the array, add `1` to its returned index.
3. Write `count_in_array_basic_r(int A[], int size, int n)`.
 - Return how many times `n` appears in the array.
4. Inside `example3_recursive_array_search`, use:

```
int A[6] = {5, 1, 5, 3, 5, 8};
```

Expected output:

```
5 in A? 1
7 in A? 0
first index of 5: 0
first index of 8: 5
count of 5: 3
```

Question 4 - Recursive Pointer Range [10 points]

Process an array using two pointers: `first` and `last`.

Requirements:

1. Write `min_p_r(int *first, int *last)`.
 - `last` points one position after the final element.
 - Base case: if `last - first == 1`, return `*first`.

- Write `array_sum_p_r(int *first, int *last)`.
 - Base case: if `first == last`, return 0.
- Write `array_copy_p_r(int *dest, int *first, int *last)`.
 - Copy values from the range `[first, last)` into `dest`.
- Inside `example4_recursive_pointer_range`, use:

```
int A[5] = {9, -4, 6, 2, -1};
int B[5];
```

Expected output:

```
min: -4
sum: 12
B: 9 -4 6 2 -1
```

Hint:

- In pointer range functions, the empty range is `first == last`.

Question 5 - Recursive String Printer [10 points]

Print and measure a string recursively.

Requirements:

- Write `print_string_basic_r(char *p)`.
 - Stop when `*p == '\0'`.
- Write `print_string_reverse_basic_r(char *p)`.
 - Stop when `*p == '\0'`.
 - To reverse the output, recurse first and print after the recursive call returns.
- Write `str_len_basic_r(char *p)`.
 - Return the number of characters before `'\0'`.
- Inside `example5_recursive_string_printer`, use:

```
char s[20] = "pointer";
```

Expected output:

```
normal: pointer
reverse: retniop
length: 7
```

Question 6 - Broken Code: multiply_r [10 points]

The function below is supposed to multiply **a** and **b** recursively without using *****. It has **one bug** that gives a result that is too large.

```
int multiply_r(int a, int b) {  
    if (a == 0) return 0;  
    return a + multiply_r(a - 1, b);  
}
```

Fix `multiply_r` and test it inside `example6_broken_multiply_r` with:

```
printf("6 * 4 = %d\n", multiply_r(6, 4));  
printf("0 * 9 = %d\n", multiply_r(0, 9));
```

Expected output after fix:

```
6 * 4 = 24  
0 * 9 = 0
```

Hint:

- Repeated addition for **a * b** means adding which value exactly **a** times?

Question 7 - Broken Code: int_division_r [10 points]

The function below is supposed to calculate integer division recursively. It has **one bug** that makes exact divisions one too small.

```
int int_division_r(int a, int b) {  
    if (a <= b) return 0;  
    return 1 + int_division_r(a - b, b);  
}
```

Fix `int_division_r` and test it inside `example7_broken_int_division_r` with:

```
printf("29 / 5 = %d\n", int_division_r(29, 5));  
printf("20 / 5 = %d\n", int_division_r(20, 5));  
printf("3 / 5 = %d\n", int_division_r(3, 5));
```

Expected output after fix:

```
29 / 5 = 5
20 / 5 = 4
3 / 5 = 0
```

Hint:

- Should the recursion stop when **a** is equal to **b**, or only when **a** is smaller than **b**?

Question 8 - Broken Code: array_max_r [10 points]

The function below is supposed to return the maximum element of an array. It has **one bug** that ignores the current first element.

```
int array_max_r(int A[], int size) {
    int max_of_rest;
    if (size == 1) return A[0];
    max_of_rest = array_max_r(A + 1, size - 1);
    return max_of_rest;
}
```

Fix `array_max_r` and test it inside `example8_broken_array_max_r` with:

```
int A[5] = {20, -2, 5, 3, 4};
printf("max: %d\n", array_max_r(A, 5));
```

Expected output after fix:

```
max: 20
```

Hint:

- After finding the maximum of the rest of the array, what should it be compared with?

Question 9 - Broken Code: count_in_array_r [10 points]

The function below is supposed to count how many times **n** appears in an array. It has **one bug** that stops after checking only the first element.

```
int count_in_array_r(int A[], int size, int n) {
    if (size == 0) return 0;
    if (A[0] == n) return 1;
```

```
    return count_in_array_r(A + 1, size - 1, n);  
}
```

Fix `count_in_array_r` and test it inside `example9_broken_count_in_array_r` with:

```
int A[6] = {5, 1, 5, 3, 5, 8};  
printf("count of 5: %d\n", count_in_array_r(A, 6, 5));  
printf("count of 7: %d\n", count_in_array_r(A, 6, 7));
```

Expected output after fix:

```
count of 5: 3  
count of 7: 0
```

Hint:

- If `A[0]` matches, should the function stop, or should it still count the rest of the array?

Question 10 - Broken Code: `print_string_reverse_r` [10 points]

The function below is supposed to print a string in reverse order. It has **one bug** that prints the string in normal order.

```
void print_string_reverse_r(char *p) {  
    if (*p) {  
        printf("%c", *p);  
        print_string_reverse_r(p + 1);  
    }  
}
```

Fix `print_string_reverse_r` and test it inside `example10_broken_print_string_reverse_r` with:

```
print_string_reverse_r("pointer");  
printf("\n");
```

Expected output after fix:

```
retniop
```

Hint:

- For reverse output, should the character be printed before or after the recursive call?